



Università degli studi di Milano

Dipartimento di Informatica

## Assignment 5

### Clustering and Initialization

*k-means vs k-means++: An Empirical Evaluation of Seeding Strategies on Clustering Performance*

Oscar Mostafa Fath el bab

Student ID: 67042A

**Course:** Statistical Methods for Machine Learning

**Professor:** Cesa Bianchi Nicolo' Antonio

June 2026

# Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

# 1 Introduction

Clustering is a fundamental unsupervised learning technique, yet the classic  $k$ -means algorithm is notoriously sensitive to its initial centroid placement. Standard uniform random initialization often leads to suboptimal local minima and high variance in the final inertia across independent runs. To mitigate this issue, the  $k$ -means++ algorithm introduces a probabilistic seeding strategy ( $D(x)^2$  sampling) designed to distribute initial centroids effectively, theoretically guaranteeing a more robust and accurate convergence.

This report presents an empirical evaluation of both initialization strategies to assess their actual impact on clustering performance. We compare standard  $k$ -means and  $k$ -means++ across three different cluster regimes (i.e.  $k \in K : |K| = 3$ ) using two distinct datasets: a 20-dimensional synthetic dataset and a 617-dimensional real-world dataset. Through a rigorous analysis of final inertia and algorithmic stability over multiple trials, this study evaluates the practical advantages and limitations of smart seeding versus pure random exploration under varying degrees of data complexity.

## 2 Datasets

To evaluate the empirical performance of both initialization strategies, two fundamentally different configurations were tested: a controlled, highly structured synthetic dataset and a complex, high-dimensional real-world perceptual dataset.

### 2.1 Synthetic Dataset

The synthetic dataset was generated to establish a controlled baseline with an explicit clustering potential, strictly following the methodology of Arthur & Vassilvitskii [1]. Specifically, it consists of  $N = 1350$  instances distributed across  $k = 27$  clusters within a  $d = 20$  dimensional space. The generation process began by selecting cluster centers uniformly at random from a hypercube of side length 700 centered at the origin, after which 50 data points were sampled around each center from a multivariate Gaussian distribution with unit variance ( $\sigma^2 = 1$ ). The exact procedural logic used to construct this clustering environment is detailed in the computational snippet below:

```
1  centers = rng.uniform(  
2      hypercube_center - hypercube_side/2,  
3      hypercube_center + hypercube_side/2,  
4      (k, d)  
5  )  
6  X = np.vstack([rng.normal(  
7      loc=c,  
8      scale=math.sqrt(variance),  
9      size=(datapoints_per_center, d)  
10     ) for c in centers])
```

According to the empirical three-sigma rule, approximately 99.7% of the data points fall within a radius of  $3\sigma$  from their respective centers. Given the unit variance ( $\sigma = 1$ ) and the vast inter-center distances, this statistically guarantees the formation of strictly disjoint, non-overlapping spherical clusters.

## 2.2 Real-World Dataset

The real-world evaluation was conducted on the **ISOLET** (Isolated Letter Speech Recognition) dataset from the UCI Machine Learning Repository. This dataset captures a complex, noisy, and perceptual task consisting of spoken letters of the English alphabet.

ISOLET is a **multiclass** problem containing 26 distinct classes (one for each letter, from A to Z) with an almost perfectly balanced distribution of approximately 300 instances per class. It is characterized by **high-dimensionality**, tracking 617 continuous, real-valued attributes per instance. These features represent a variety of acoustic properties extracted from speech signals, including spectral coefficients, contour features, and sonorant features.

## 2.3 Preprocessing and Merging

For the synthetic dataset, no preprocessing was required, as the features were intrinsically generated with the desired spatial configuration.

Regarding the real dataset, since clustering is an unsupervised learning task that does not require a train/test paradigm, the original training split (`isolet1+2+3+4.data`, 6,238 instances) and the testing split (`isolet5.data`, 1,559 instances) were concatenated into a single cohesive matrix (`isolet.data`) containing a total of 7,797 instances. The final column representing the class labels was entirely discarded prior to execution, ensuring the clustering partition relied purely on feature distances.

Finally, to account for the inherent heterogeneity of the acoustic attributes, which measure physical quantities of fundamentally different natures, the continuous features of the ISOLET dataset were standardized. This ensures that no single attribute disproportionately influences the Euclidean distance computations. Specifically, each of the 617 attributes was scaled to have a mean of 0 and a variance of 1.

# 3 Methodology

## 3.1 Algorithms

### 3.1.1 $k$ -means

The standard  $k$ -means algorithm partitions a dataset of  $N$  datapoints in a  $d$ -dimensional space into  $k$  distinct clusters by iteratively minimizing the within-cluster sum of squares (inertia). The process simply alternates between assigning each point to the nearest centroid and recomputing the centroids as the mean of their assigned points, halting when assignments no longer change.

Translating this theoretical simplicity into reliable code requires strict algorithmic **completeness**. To achieve this, the core execution was protected by an input validation subroutine. This step intercepts edge cases—such as non-numeric arrays, invalid cluster budgets ( $k \leq 0$  or  $k > N$ ), and dimensional mismatches—guaranteeing that the algorithm handles arbitrary inputs gracefully without unexpected runtime crashes.



```

1  def _validate_inputs(k, X):
2      if X is None or X.size == 0:
3          raise ValueError("Error: Dataset X is empty.")
4      if k <= 0:
5          raise ValueError(f"Error: k must be greater than 0. Received k={k}.")
6      if k > X.shape[0]:
7          raise ValueError(
8              f"Error: k ({k}) cannot be greater than the number of \
9              data points in the dataset ({X.shape[0]})."
10         )
11     if np.isnan(X).any() or np.isinf(X).any():
12         raise ValueError(
13             "Error: Dataset X contains NaN (Not a Number) or Infinite \
14             values."
15         )

```

Combining this defensive validation with the iterative optimization loop, the complete  $k$ -means algorithm looks as follows:

```

1  def _inner_kmeans(k, X, centroids):
2      while True:
3          distances = np.linalg.norm(X[:, np.newaxis, :] - \
4          centroids[np.newaxis, :, :], axis=2)
5          labels = np.argmin(distances, axis=1)
6          new_centroids = np.array([
7              X[labels == i].mean(axis=0) if np.any(labels == i) else centroids[i]
8              for i in range(k)
9          ])
10         if np.allclose(centroids, new_centroids): break
11         centroids = new_centroids
12     return centroids, distances, labels
13
14  def kmeans(k, X):
15      _validate_inputs(k, X)
16      return _inner_kmeans(k, X,
17          X[rng.choice(X.shape[0], size=k, replace=False)]
18      )

```

### 3.1.2 $k$ -means++

The  $k$ -means++ variant addresses the inherent vulnerability of the standard algorithm to poor initializations. Instead of selecting all initial centroids uniformly at random, it introduces a probabilistic, distance-based seeding phase. The first centroid is chosen uniformly at random, while each subsequent centroid is selected with a probability strictly proportional to its squared Euclidean distance from the nearest already-chosen centroid (a technique known as  $D^2$ -sampling). Once the  $k$  initial centroids are established, the algorithm delegates the optimization to the standard iterative loop.

Implementing  $D^2$ -sampling requires handling cases where the sum of minimum squared distances evaluates to zero, which occurs if unassigned datapoints perfectly overlap with the chosen centroids. To prevent division-by-zero errors during probability calculation and maintain algorithmic **completeness**, the implementation falls back to a uniform probability distribution to select the remaining centroids.

```

1  def kmeans_plus_plus(k, X):
2      _validate_inputs(k, X)
3      centroids = [X[rng.choice(X.shape[0])]]
4      D2 = np.inf
5      for _ in range(1, k):
6          D2 = np.minimum(D2, np.sum((X - centroids[-1])**2, axis=1))
7          sum_D2 = np.sum(D2)
8          probs = D2 / sum_D2 if sum_D2 > 0 else np.ones(X.shape[0]) / X.shape[0]
9          centroids.append(X[rng.choice(X.shape[0], p=probs)])
10     return _inner_kmeans(k, X, np.array(centroids))

```

## 3.2 Metrics and graphics

To evaluate the initialization strategies, both  $k$ -means and  $k$ -means++ were executed on the synthetic and ISOLET datasets across multiple values of  $k$ . Given the stochastic nature of centroid initialization, a fixed number of independent trials was performed for each specific configuration to ensure statistical validity.

To numerically assess performance and computational trade-offs, four key metrics were aggregated across the trials: **mean inertia**, **standard deviation of inertia**, **minimum inertia**, and **mean execution time**. Additionally, the percentage improvement of  $k$ -means++ over the standard algorithm was explicitly calculated for all four metrics to quantify the practical impact of the  $D^2$ -sampling.

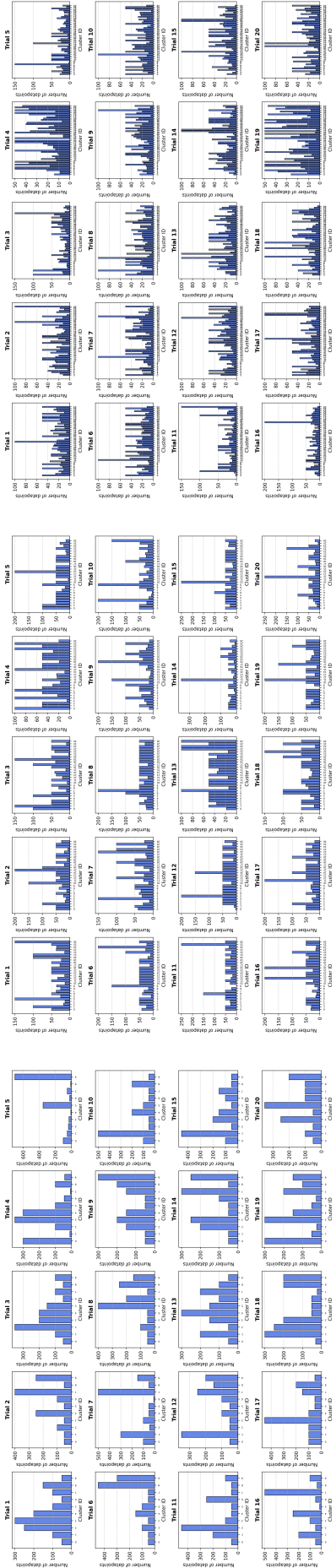
Finally, to complement the numerical evaluation, two visualization strategies were employed: **bar charts** representing the exact cluster size distributions (number of points assigned per cluster), and side-by-side **boxplots** mapping the final inertia distributions to visually contrast the stability and variance of the two algorithms across the independent runs.

## 4 Experiments and Conclusions

This section presents the empirical results obtained through a systematic evaluation of both initialization strategies. For each of the two datasets, both the standard  $k$ -means and  $k$ -means++ algorithms were rigorously tested across three distinct cluster budgets ( $k = 10, 27$  and  $50$  for the synthetic dataset and  $k = 10, 26$  and  $50$  for the real dataset), executing 20 independent trials for each specific scenario. This experimental setup corresponds to an explicit combinatorial framework of:

$$2 \text{ datasets} \times 2 \text{ algorithms} \times 3 \text{ values of } k \times 20 \text{ trials} = 240 \text{ total executions.}$$

The aggregated metrics and visual distributions resulting from these 240 runs are analyzed below.

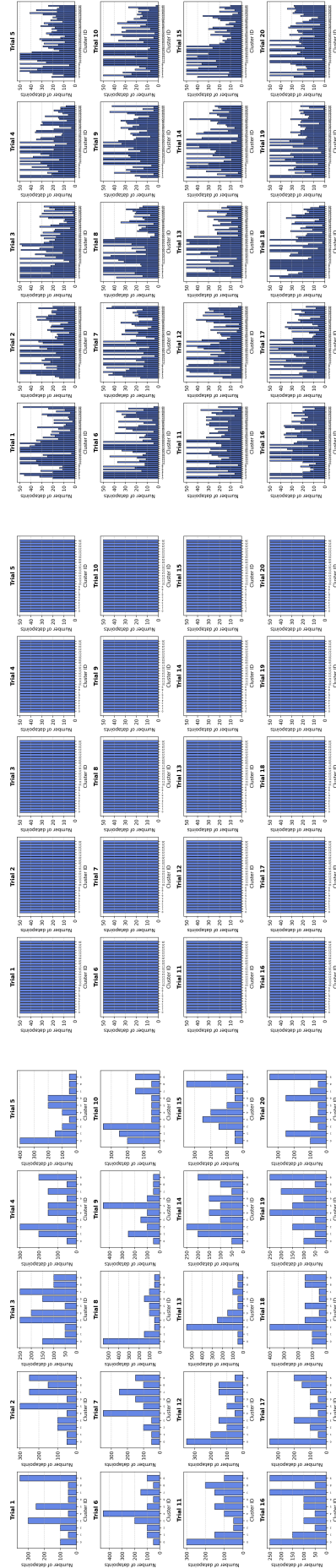


(a)  $k = 10$

(b)  $k = 27$

(c)  $k = 50$

Figure 1: Cluster distribution obtained via standard  $k$ -means on the synthetic dataset.

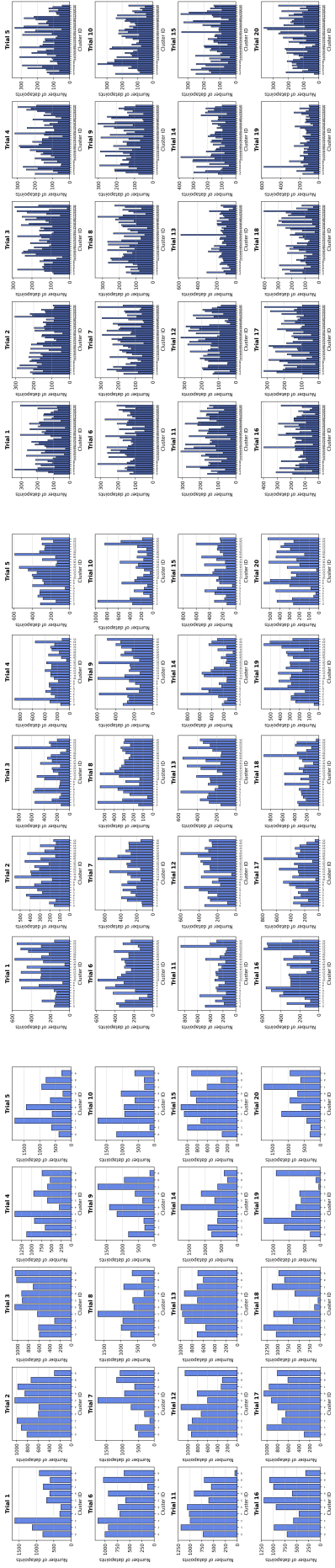


(a)  $k = 10$

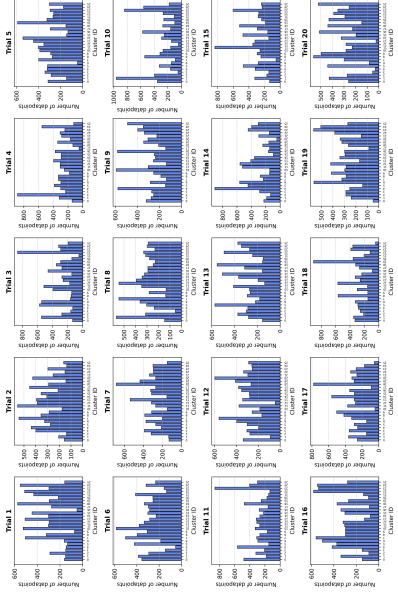
(b)  $k = 27$

(c)  $k = 50$

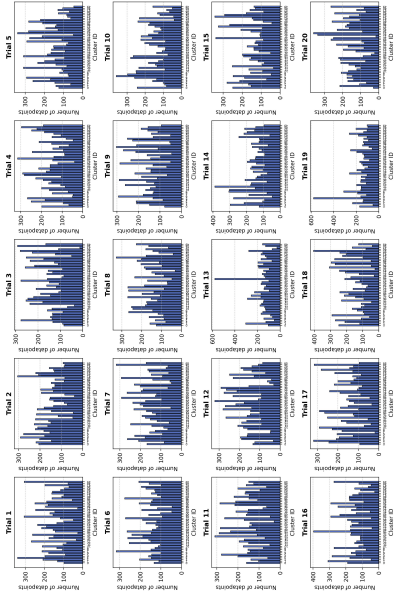
Figure 2: Cluster distribution obtained via  $k$ -means++ on the synthetic dataset.



(a)  $k = 10$

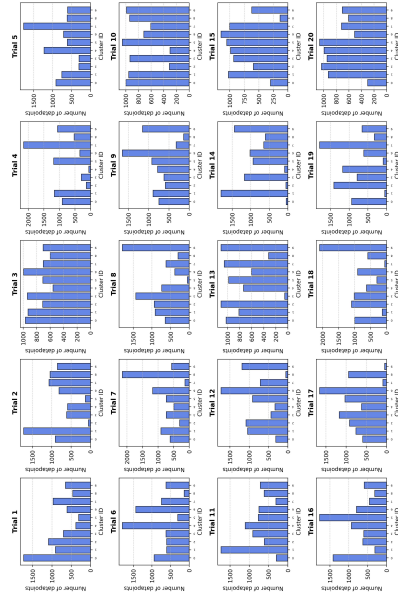


(b)  $k = 26$

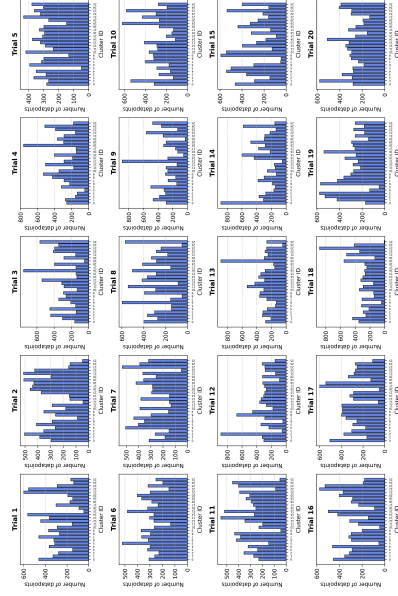


(c)  $k = 50$

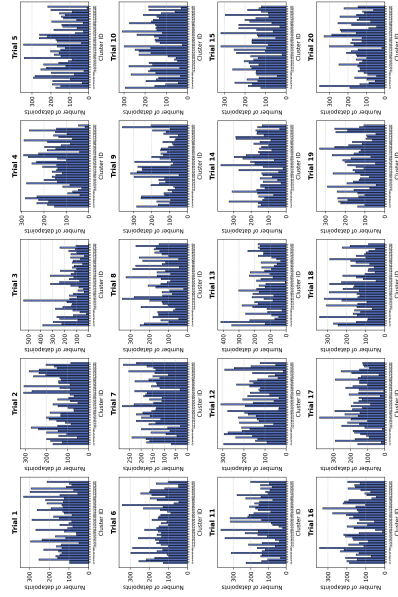
Figure 3: Cluster distribution obtained via standard  $k$ -means on the real dataset (ISOLET).



(a)  $k = 10$



(b)  $k = 26$



(c)  $k = 50$

Figure 4: Cluster distribution obtained via  $k$ -means++ on the real dataset (ISOLET).

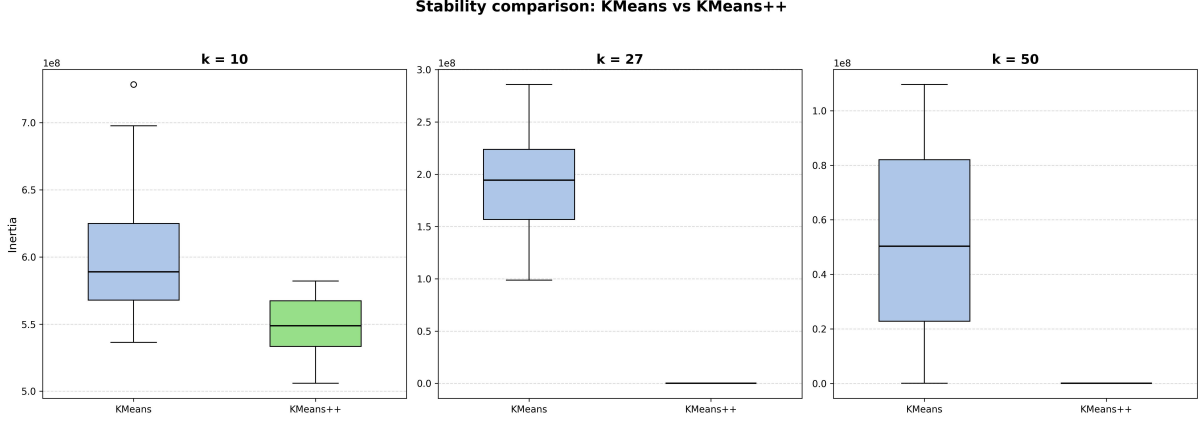


Figure 5: Inertia distribution on the **Synthetic Dataset** for  $k \in \{10, 27, 50\}$ .

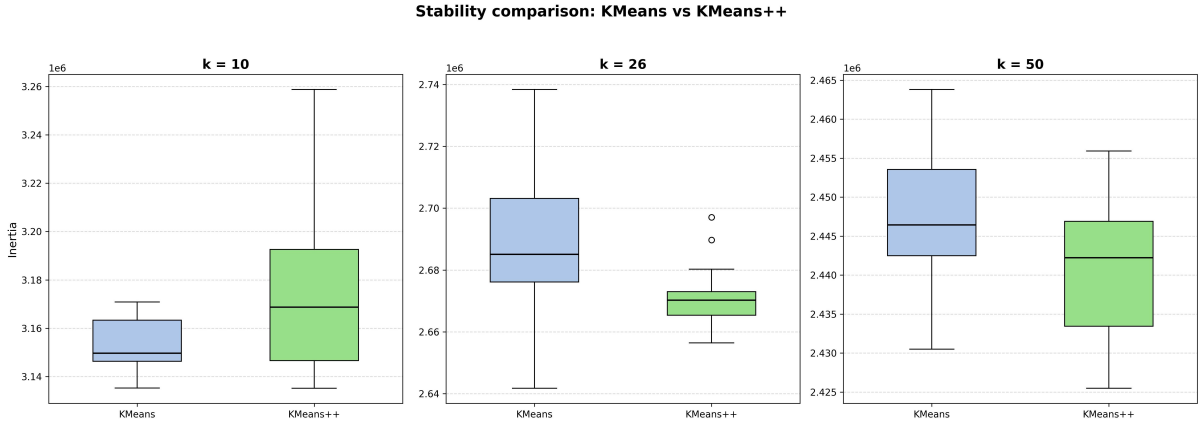


Figure 6: Inertia distribution on the **ISOLET Dataset** for  $k \in \{10, 26, 50\}$ .

| $k$ | Avg Time (s) |         | Avg Inertia          |         | Inertia Std Dev      |          | Min Inertia          |         |
|-----|--------------|---------|----------------------|---------|----------------------|----------|----------------------|---------|
|     | Value        | Impr.   | Value                | Impr.   | Value                | Impr.    | Value                | Impr.   |
| 10  | 0.0061       | +26.76% | $5.9849 \times 10^8$ | +8.35%  | $4.8286 \times 10^7$ | +59.76%  | $5.3631 \times 10^8$ | +5.67%  |
| 27  | 0.0192       | +52.82% | $1.9129 \times 10^8$ | +99.99% | $4.6869 \times 10^7$ | +100.00% | $9.8752 \times 10^7$ | +99.97% |
| 50  | 0.0594       | +11.71% | $5.2427 \times 10^7$ | +99.95% | $3.1767 \times 10^7$ | +100.00% | $2.4999 \times 10^4$ | +0.65%  |

Table 1: Absolute performance metrics and percentage improvements of  $k$ -means++ over standard  $k$ -means on the **Synthetic Dataset** across 20 trials.

| $k$ | Avg Time (s) |         | Avg Inertia          |        | Inertia Std Dev      |          | Min Inertia          |        |
|-----|--------------|---------|----------------------|--------|----------------------|----------|----------------------|--------|
|     | Value        | Impr.   | Value                | Impr.  | Value                | Impr.    | Value                | Impr.  |
| 10  | 4.3109       | +3.34%  | $3.1530 \times 10^6$ | -0.70% | $1.2063 \times 10^4$ | -191.46% | $3.1353 \times 10^6$ | +0.01% |
| 26  | 13.0302      | +2.06%  | $2.6853 \times 10^6$ | +0.53% | $2.4727 \times 10^4$ | +62.32%  | $2.6417 \times 10^6$ | -0.56% |
| 50  | 22.3064      | -14.11% | $2.4472 \times 10^6$ | +0.28% | $9.0902 \times 10^3$ | +3.10%   | $2.4305 \times 10^6$ | +0.21% |

Table 2: Absolute performance metrics and percentage improvements of  $k$ -means++ over standard  $k$ -means on the **ISOLET Dataset** across 20 trials.

**Cluster Distribution Analysis.** An examination of the bar charts (Fig. 1-2) highlights the clear advantage of smart seeding under ideal conditions. On the synthetic dataset, by setting  $k = 27$  (equal to the true number of generated centers), *k-means++* achieves a perfect and perfectly stable partitioning across all trials (exactly 50 datapoints per cluster), whereas standard k-means fails by falling into suboptimal local minima. For mismatched values of  $k$  (10, 50), the distributions appear chaotic for both algorithms due to geometric misalignment. In the real-world ISOLET dataset, cluster distributions appear irregular for any value of  $k$ . This behavior is fully justified by the nature of the dataset: the extremely high dimensionality ( $d = 617$ ) and the acoustic-perceptual nature of the data generate highly overlapping natural clusters, preventing the formation of distinct spherical partitions.

**Inertia and Algorithmic Stability.** The analysis of the boxplots (Fig. 3-4) and metrics (Tab. 1-2) confirms the theoretical superiority of  $D^2$ -sampling. On the synthetic dataset, *k-means++* drastically reduces the inertia variance (minimizing it for  $k \geq 27$ ) and ensures a significantly lower median inertia, confirming a robust convergence towards the global optimum. On the ISOLET dataset, *k-means++* maintains its advantages (lower dispersion and reduced median inertia) for values of  $k$  appropriate to the problem’s complexity ( $k \geq 26$ , the actual number of classes). However, an interesting counterintuitive anomaly emerges for  $k = 10$ : *k-means++* proves to be less stable than standard k-means, recording an increase in both dispersion and median inertia. This phenomenon is attributable to the topology of the real-world data: by forcing the algorithm to group 26 overlapping classes into only 10 clusters, the distance-proportional sampling ( $D^2$ ) tends to hook onto outliers in the high-dimensional space as initial centroids, leading to worse local minima compared to purely random exploration.

**General Pros and Cons.** In conclusion, transitioning to *k-means++* offers clear advantages: it guarantees excellent stability and minimizes inertia when clusters are well-separated or when the  $k$  budget is appropriately sized. Conversely, the only empirical drawback observed concerns high-dimensional real-world data with significant overlap: in these scenarios, if the parameter  $k$  is severely underestimated, the strategy of forcibly distancing centroids can prove counterproductive, favoring the selection of unstable outlier data.

## References

- [1] D. Arthur and S. Vassilvitskii, *k-means++: The advantages of careful seeding*, in Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA ’07), 2007, pp. 1027–1035.
- [2] M. Fanty and R. Cole, *ISOLET Dataset*, UCI Machine Learning Repository, 1994. [Online]. Available: <https://archive-beta.ics.uci.edu/dataset/54/isolet>